

Data Versioning

Git For Your Data!

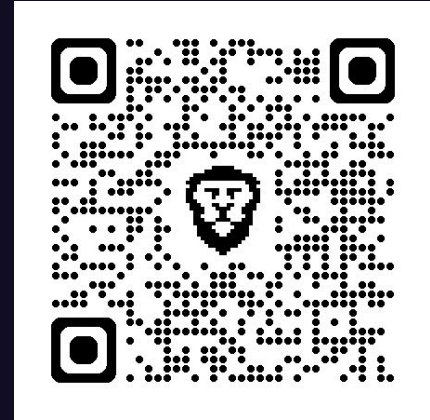
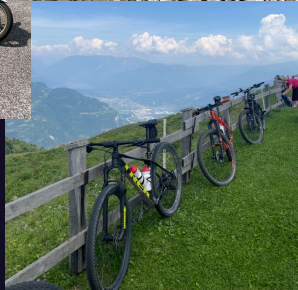
Federico Marchesi - Software Engineer
@ Prima Assicurazioni

► About me :)

My main responsibilities are:

- Trying to not make production crash

My Hobbies:



▶ “No man ever steps in the same river twice, for it's not the same river and he are not the same man. - Heraclito

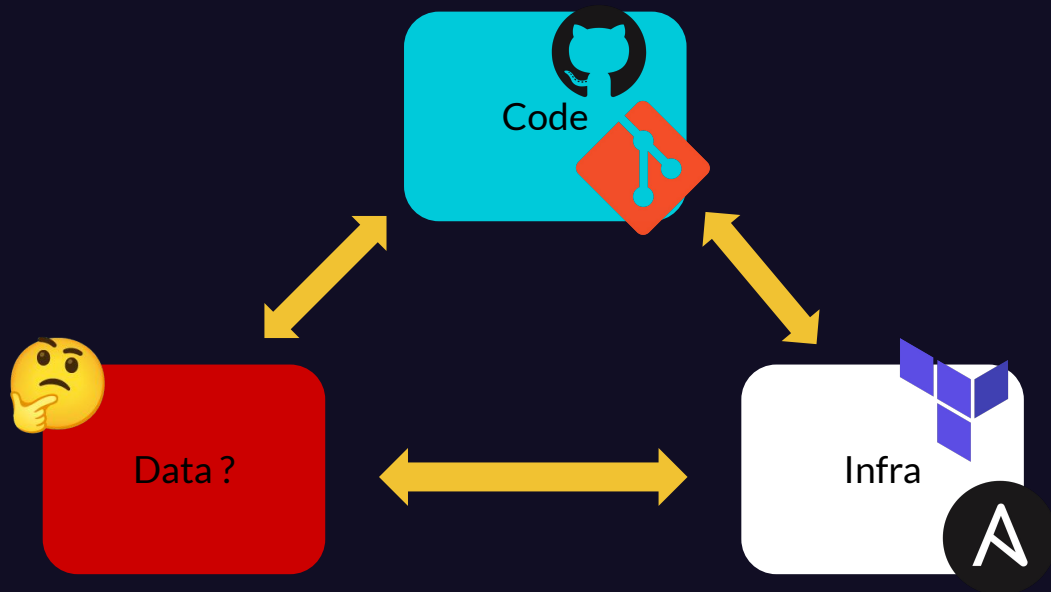


► Why we want to version data ?

Some reasons:

- Disaster recovery
- Audit trail
- CI/CD
- MLOps
- Single point of truth/Reproducibility
- Observability
- Compliance & Governance
- Debugging/Testing

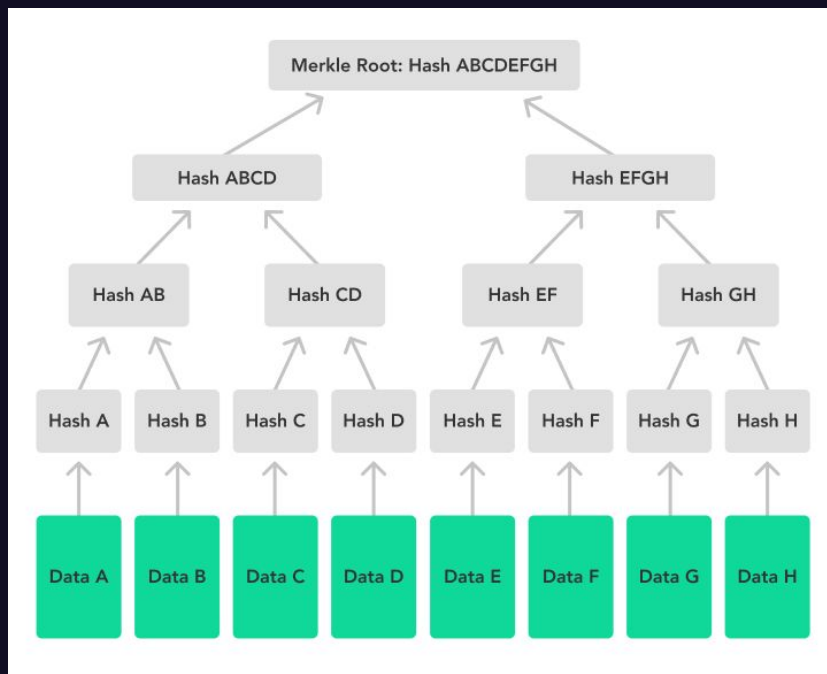
► Versioning



► Merkle trees

Merkle Trees: The Foundation of Git

- Merkle Trees are the core data structure powering Git's internal logic.
- Git views the filesystem through this lens: every file (**blob**) and directory (**tree**) is uniquely identified by a content-hash.
- This enables instant change detection. By comparing top-level hashes, Git identifies modified data immediately without scanning the entire project.



blob name → SHA-1

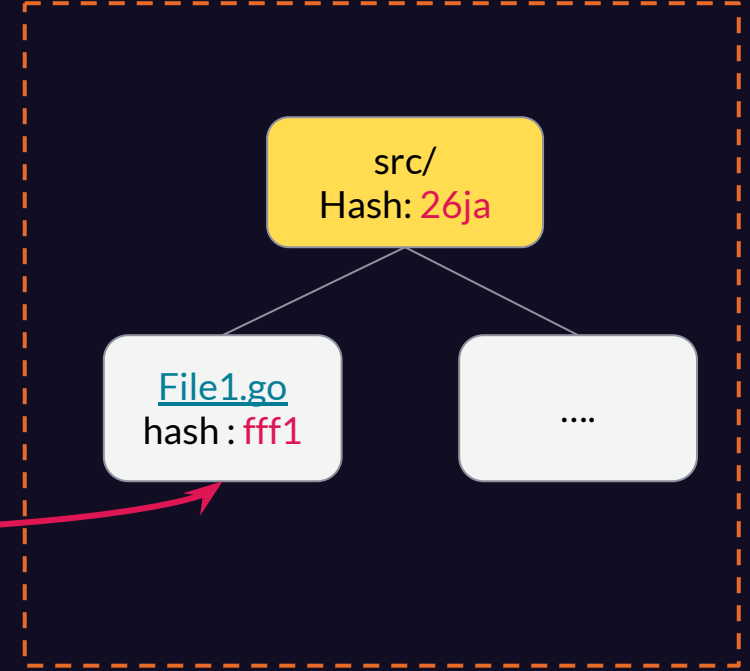
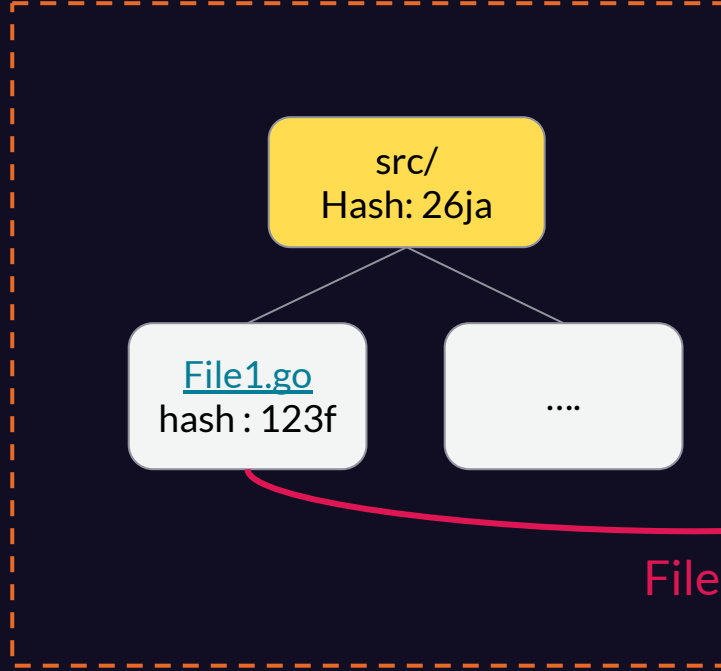
2hd82kasd89234...



```
const pluckDeep = key => obj => key.split('.').reduce((accum, key) => accum[key], obj)
const compose = (...fns) => res => fns.reduce((accum, next) => next(accum), res)

const unfold = (f, seed) => {
  const go = (f, seed, acc) => {
    const res = f(seed)
    return res ? go(f, res[1], acc.concat([res[0]])) : acc
  }
  return go(f, seed, [])
}
```

Content compressed with zlib



File change

► Git is enough for data versioning ?

Pros:

- Structural data sharing

Cons:

- Does not scale well for data formats other than text files
- Decentralized*
- Copies the structure folder, unstructured and semi-unstructured data could be cumbersome..

► How to bridge the gap with Data

Naive/Files based approach

- Git LFS
- Naive approaches

SQL /Tables

- Table versioning / SCD Type 2/4

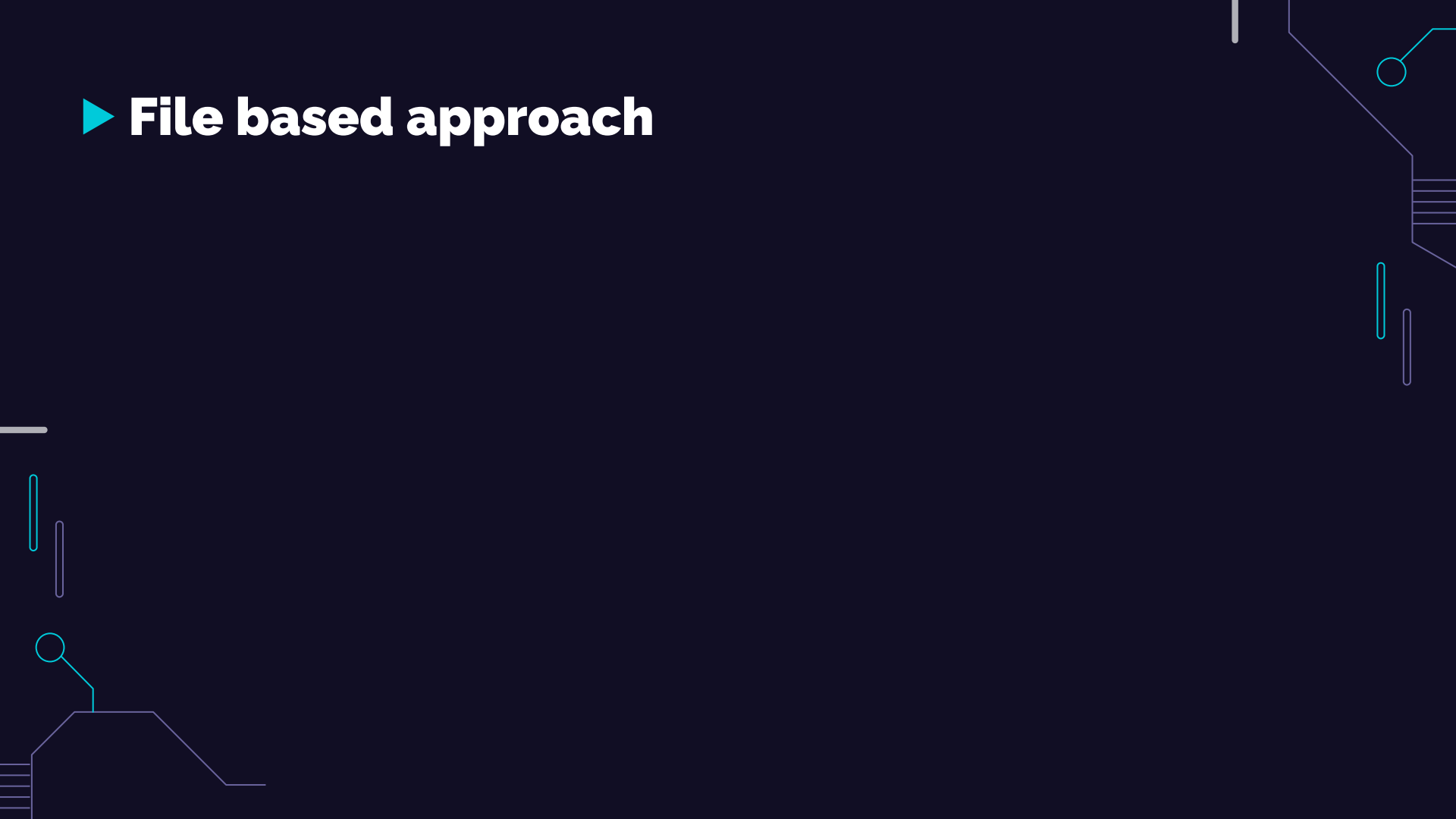
New data structure

- Prolly trees - Dolt
- Apache Iceberg

Modern Data Lakehouse

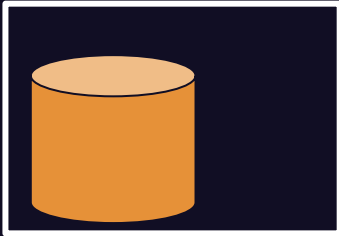
- Catalogs

► File based approach

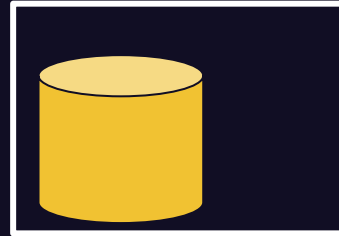


► Naive approach - Copy the full data

/data_20_02_2025...



/data_26_02_2025...



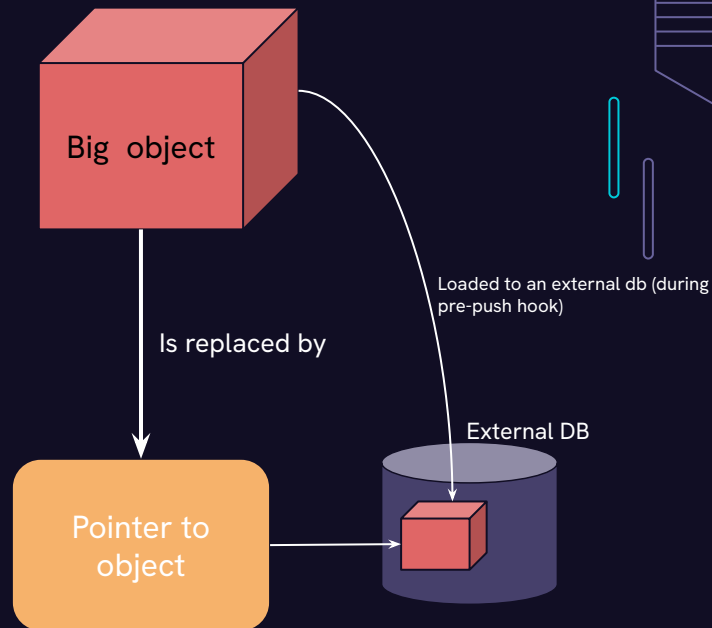
► Git LFS

Git uses **hooks** to perform operations during/before/after commits.

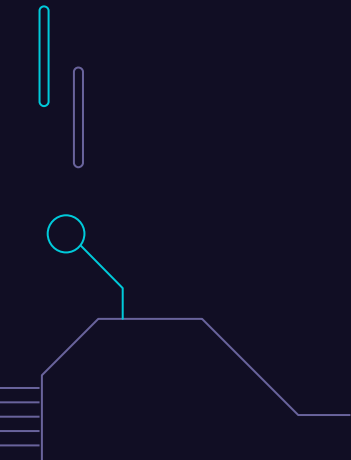
Filtered files are “replaced” by pointer files that reference the actual content.

- **pre-push**: uploads large files to the LFS server.
- **post-checkout** and **post-merge**: download the real files when switching branches or cloning the repository.

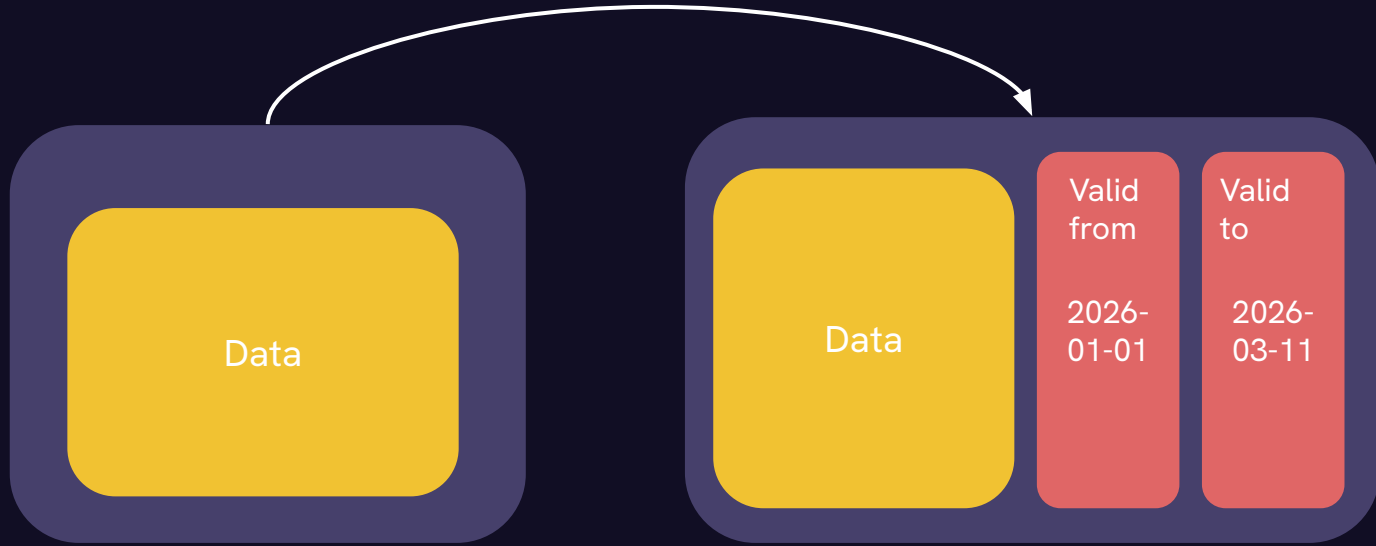
```
> git lfs track "*.bin"
> git add .gitattributes
```



► Data versioning on top of SQL



► Temporal Tables/History Tables



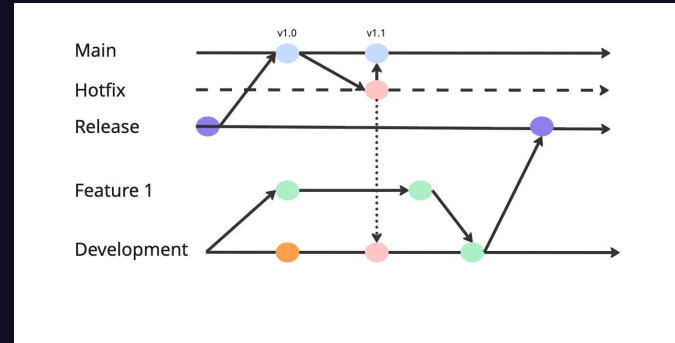
► Is this enough for real data versioning ?

Not quite.. they are al workaround:

- We are stuck into 1 dimension
- We miss branch, diff, merges rebase tagging and many other stuff
- We miss **structural data sharing**

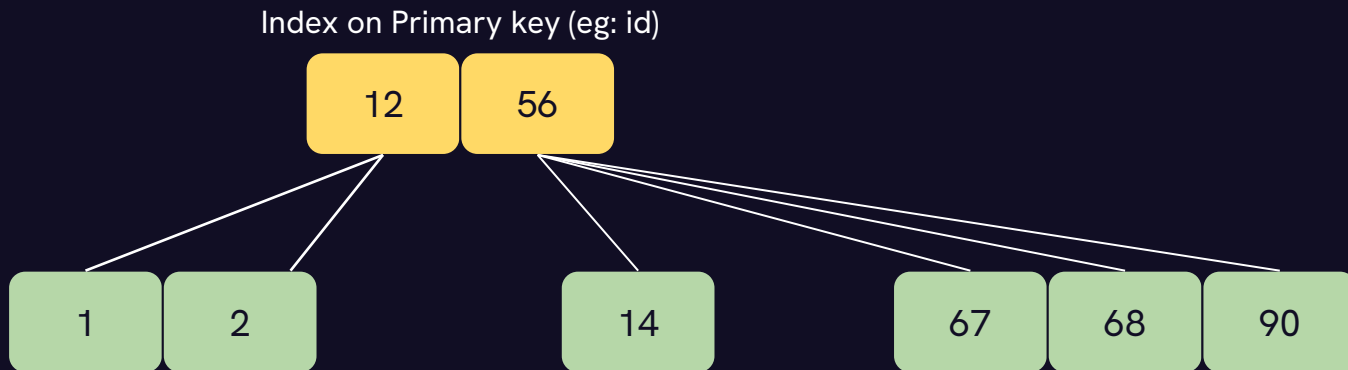


VS



► **We are trying to solve a Data Problem, with
no Data Structure...**

► B-Trees

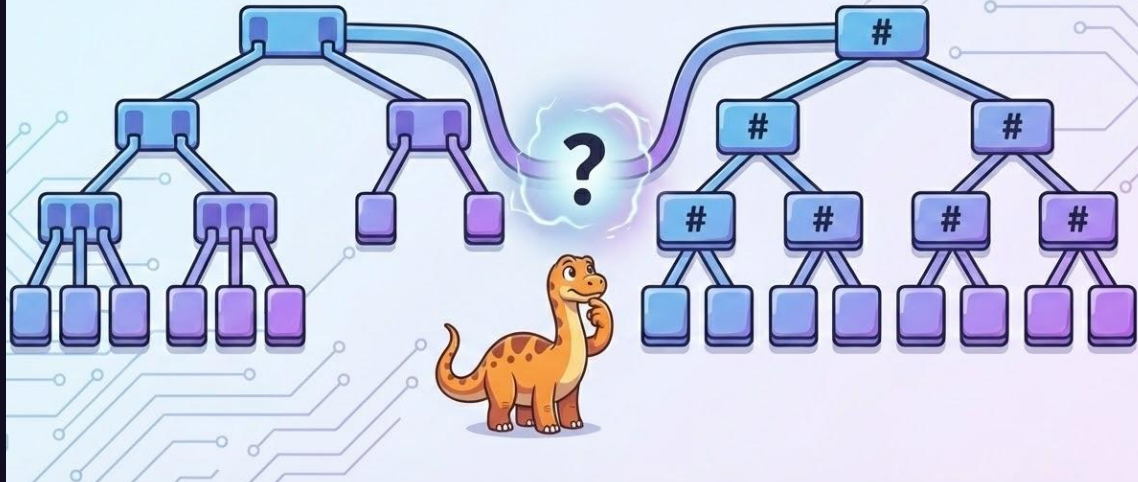


B-trees are self-balancing tree data structures that keep data sorted and allow fast searches, insertions, and deletions.

They are the **backbone** for database.

► What if ...

B-trees + Merkle Trees = ?

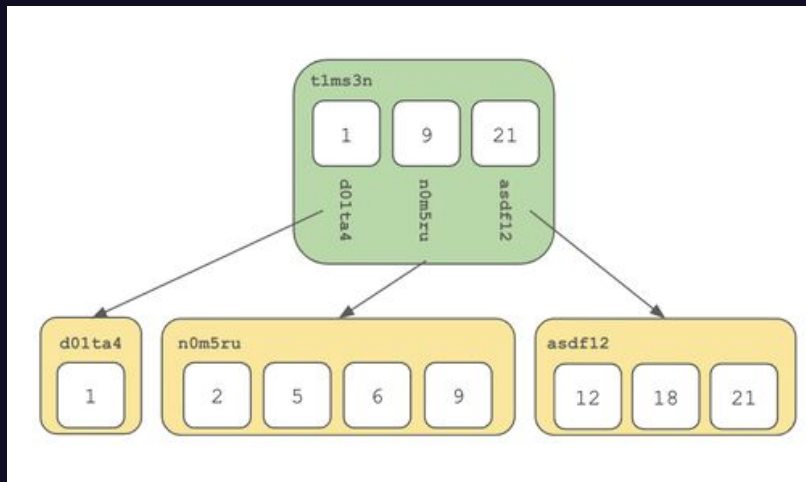


► Dolt (Proly trees)

Data structure closely related to the b-tree, more specifically could be seen as a hybrid between b-tree and merkle tree.

Invented by Noms creator and then later refined by Dolt

Proly trees use hashes of immutable content blocks as addresses to construct a data structure that enables efficient retrieval performance, support structural sharing and allows fast diff operations

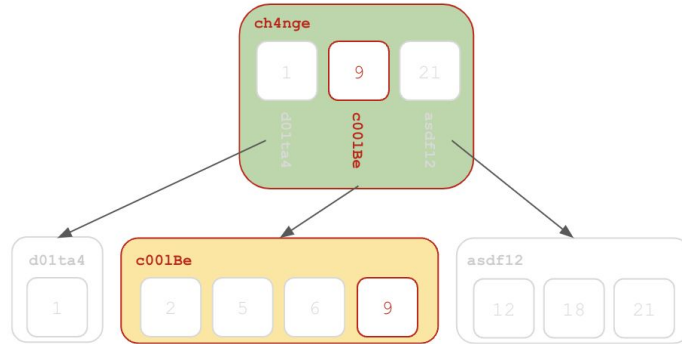


► The structural data sharing

Updated value identified by primary key 9.

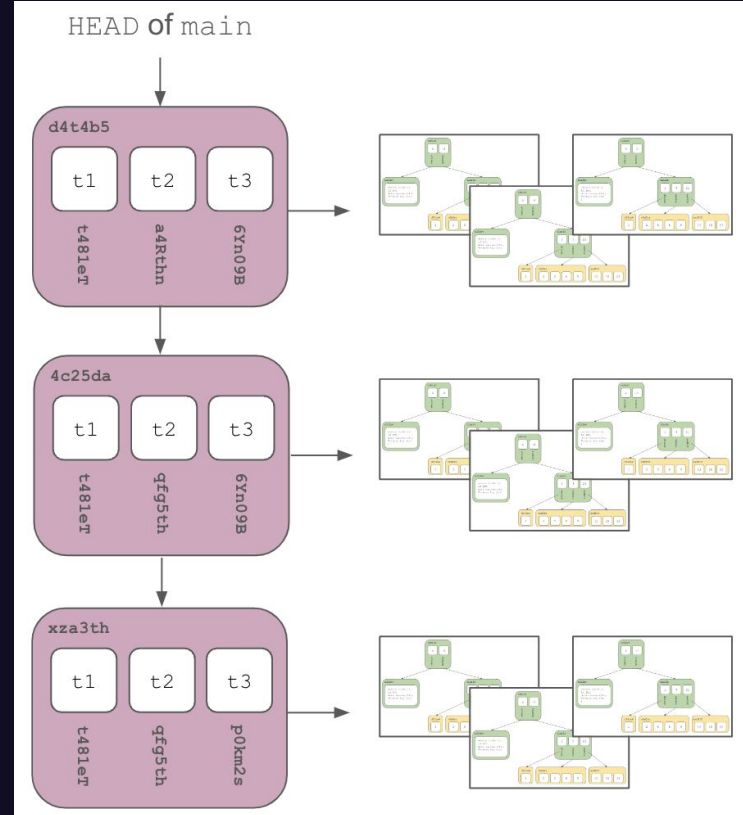
Keys and **hashes** modified colored in red.
Note, only 1 of three internal nodes changed.

Unchanged nodes grayed out for emphasis.



► Git graph tree

This means we can hash tables and create commits to retrieve the data state at any point in time. We can diff, compare, branch, and merge!



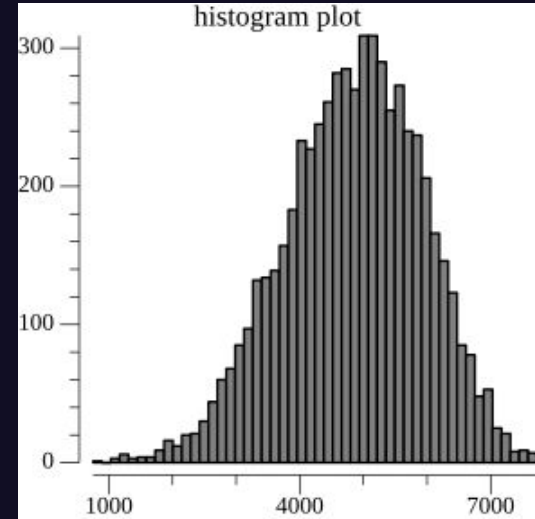
► Why it is probabilistic ?

PDF
$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2\right)$$

CDF
$$F(x) = \int_{-\infty}^x f(t) dt$$

Dolt $\rightarrow$
$$p = \frac{CDF(x + \Delta) - CDF(x)}{1 - CDF(x)}$$

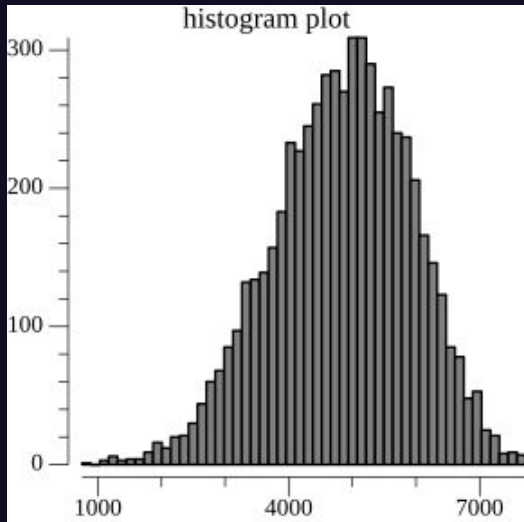
$$P(A | B) = \frac{P(A \cap B)}{P(B)}$$



► Deep Dive

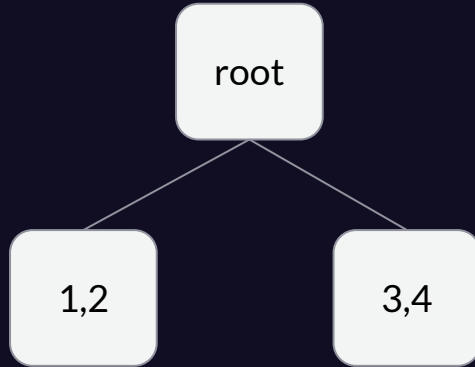
The real difference between a B-tree and a Prolly tree lies in the chunking size and splitting logic.

- A **B-tree** uses fixed-size page blocks, usually matching the OS page size to be hardware-friendly and faster.
- A **Prolly tree**, on the other hand, does not use a fixed-size chunking technique. Instead, it relies on a **rolling hash function** combined with a **Cumulative Distribution Function (CDF)**; this forces the chunk distribution to be as consistent as possible by probabilistically deciding when to split a chunk based on its content.

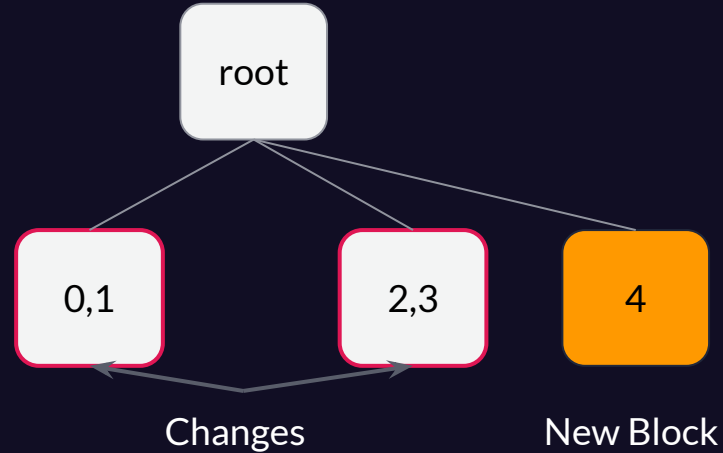
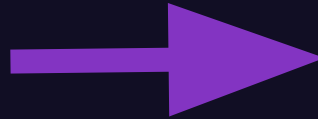


► Quick example b-tree vs prolly-tree

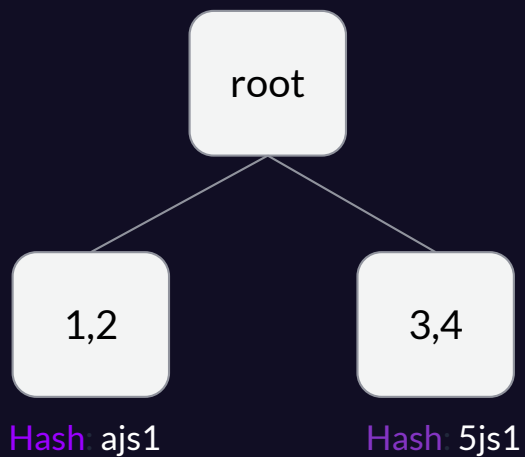
B-tree



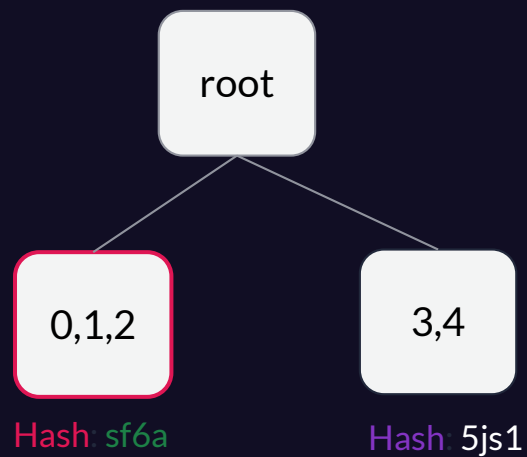
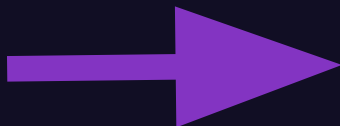
Let's insert '0'



Prolly-tree

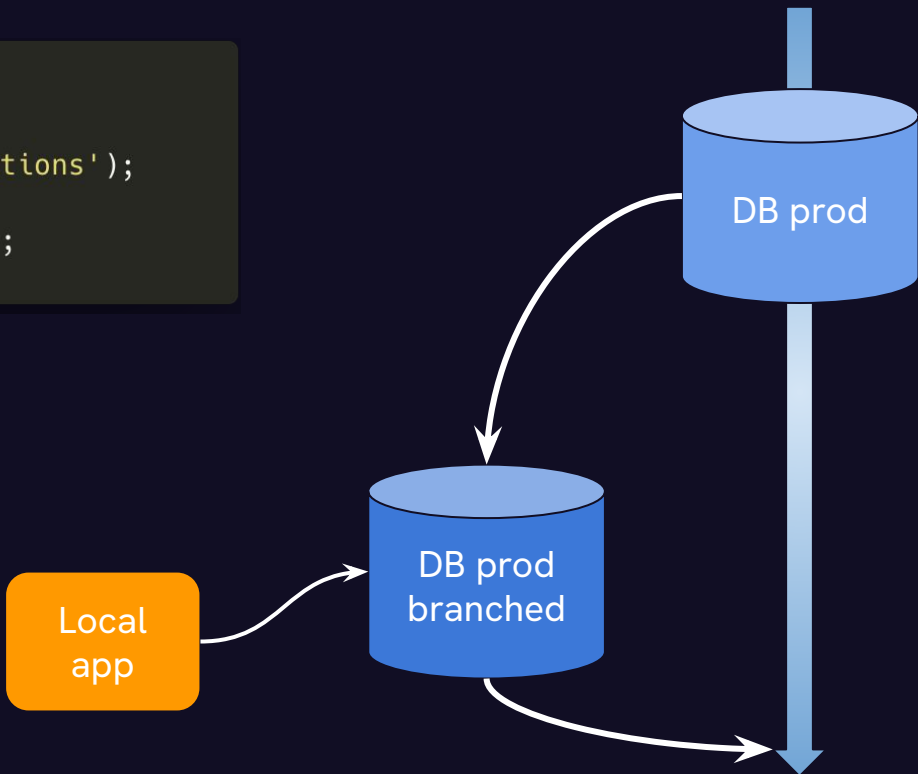


let's insert '0'

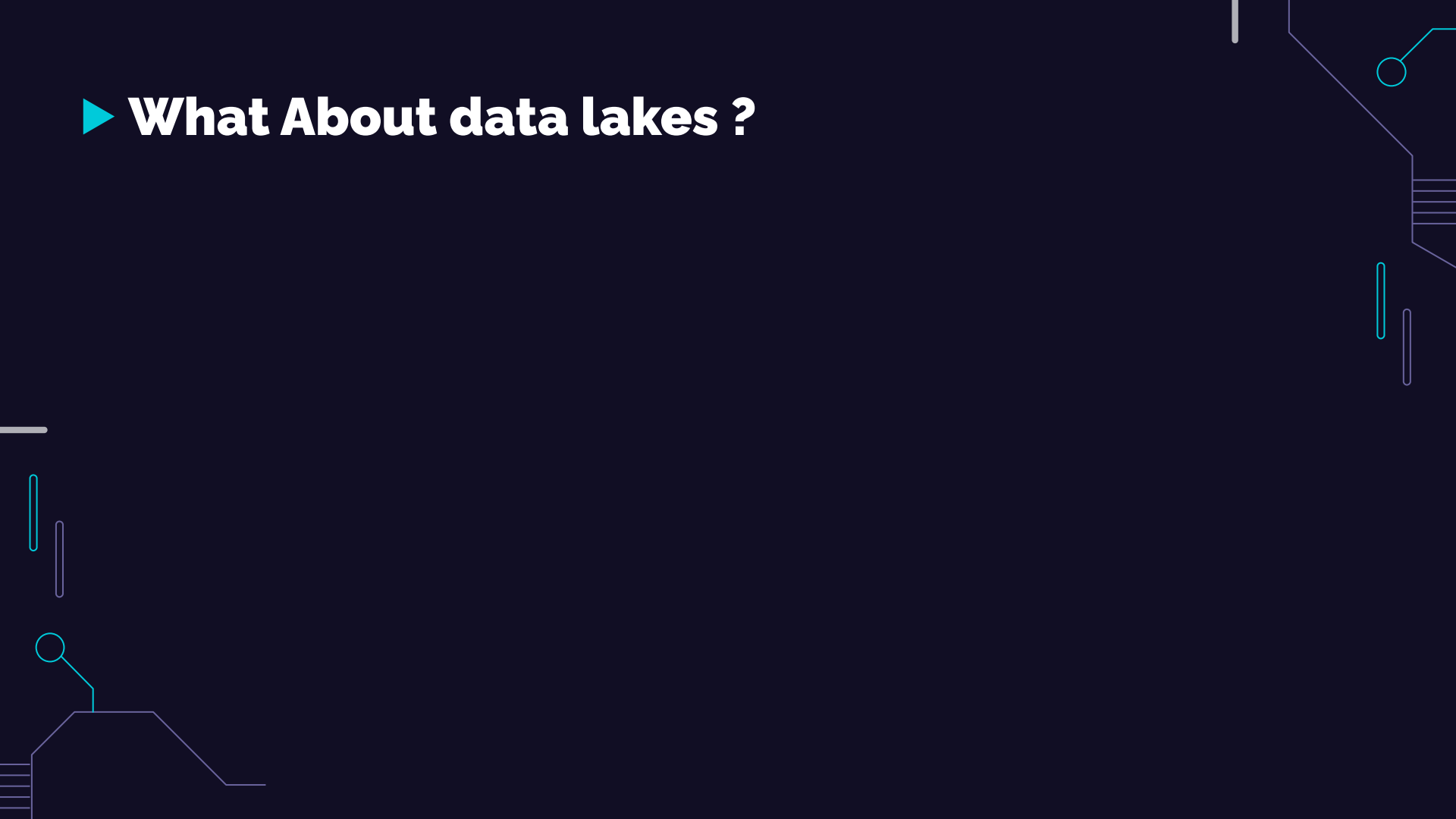




```
call dolt_checkout('-b','modifications');  
call dolt_merge('schema_changes');
```



► What About data lakes ?



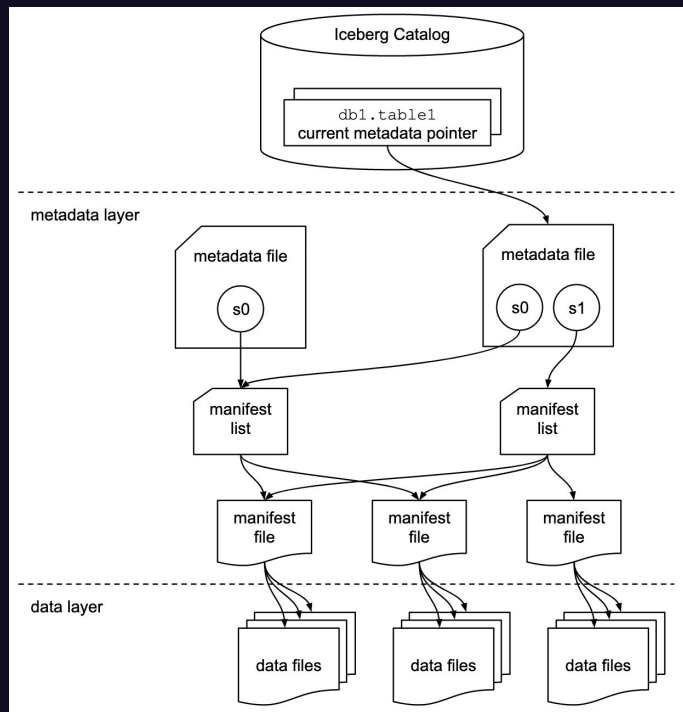
► Apache Iceberg

> Tabular format for high performance analytics

What is cool about it?

- > How it manages metadata!
- > ACID compliance
- > Enables Snapshots/time travel/Versioning
- > Hidden Partitioning
- > Schema evolution
- > MoR and MoW support

Together with a proper Catalog it enables full git-data experience



► Apache Iceberg + Nessie

Apache iceberg combined together with a full data catalog like Nessie.

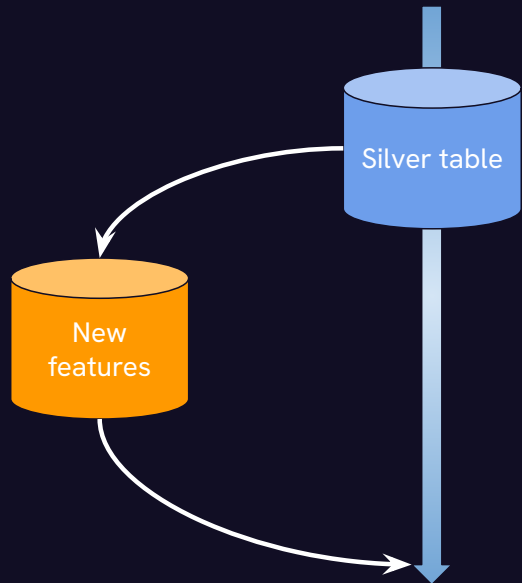
Will allow us to enable data versioning also on big data!



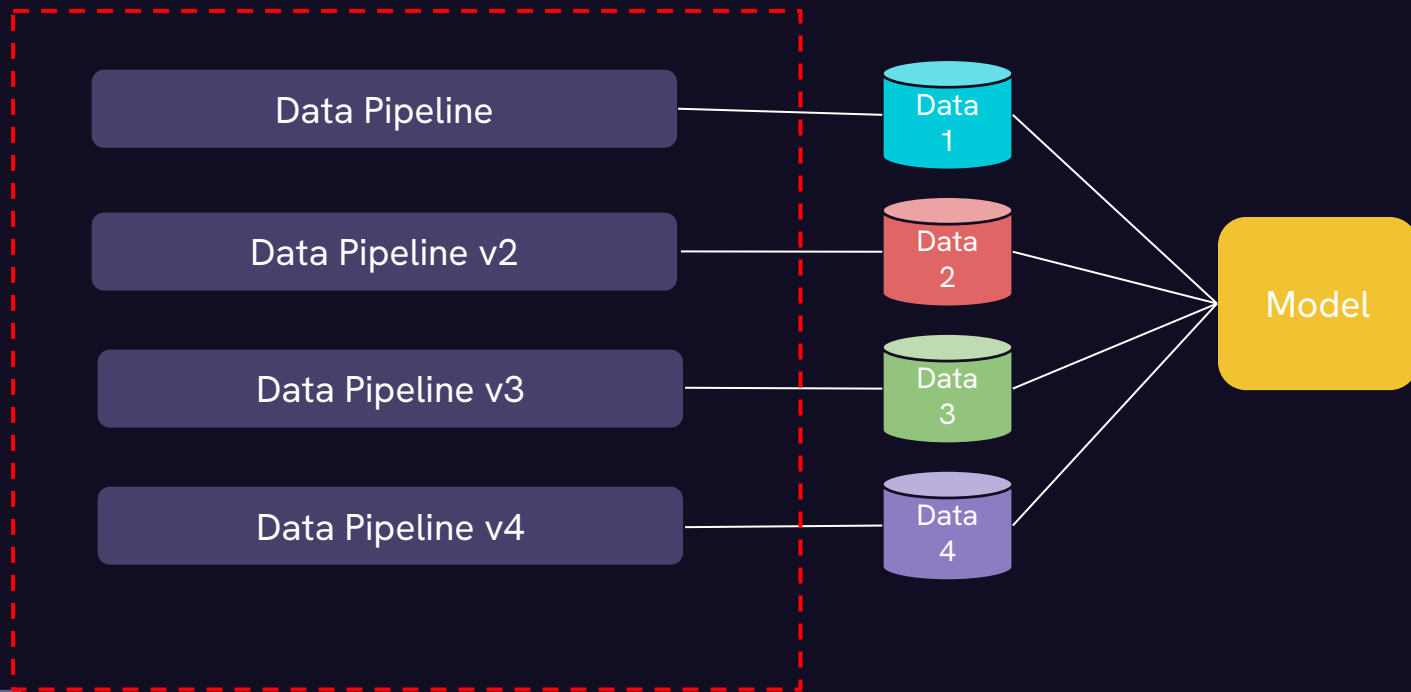
+



=



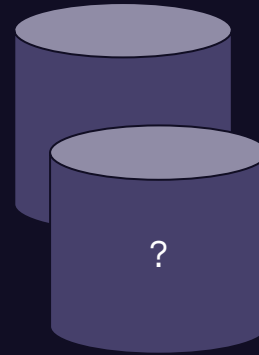
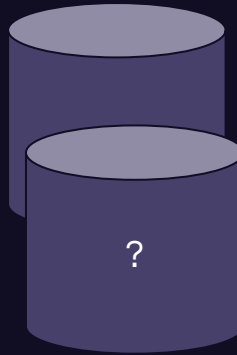
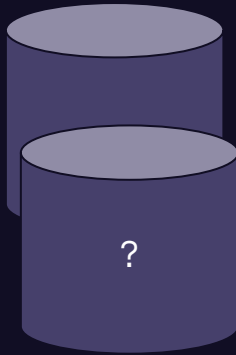
► Common ML scenario



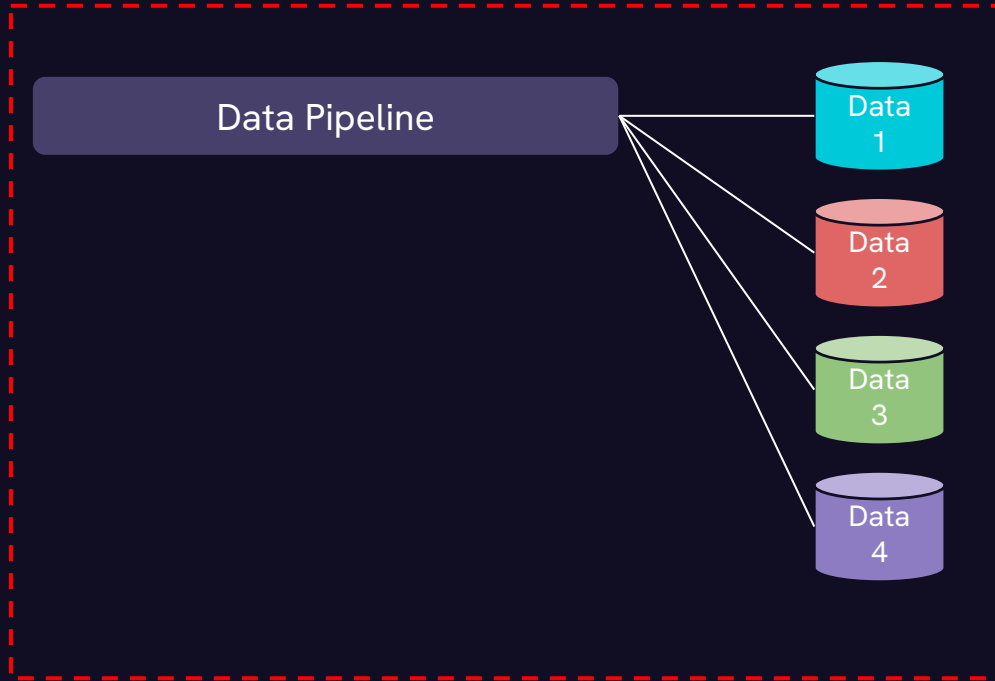
► Common situation

Which one is which?

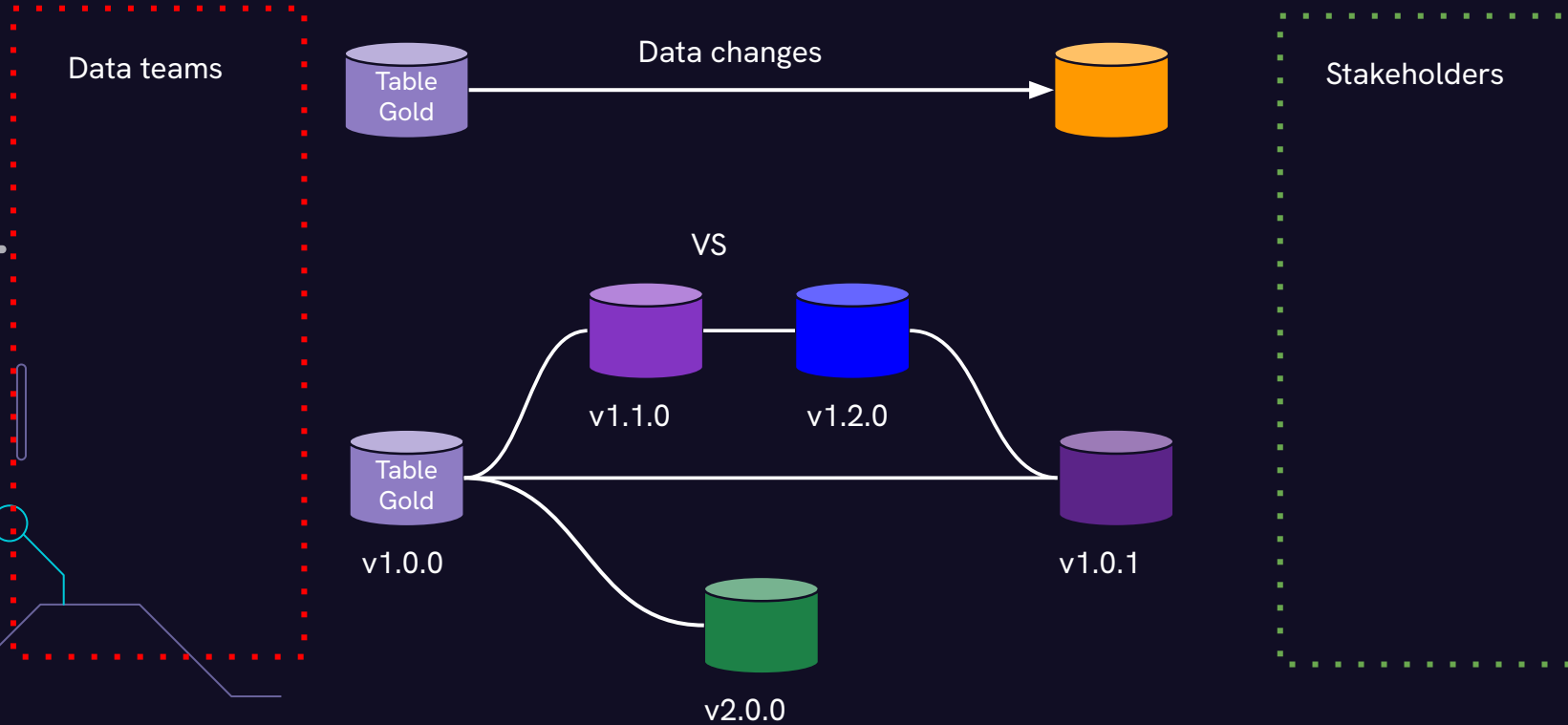
- final_v1?
- final_v1_1_final?
- final_v1_final_1?



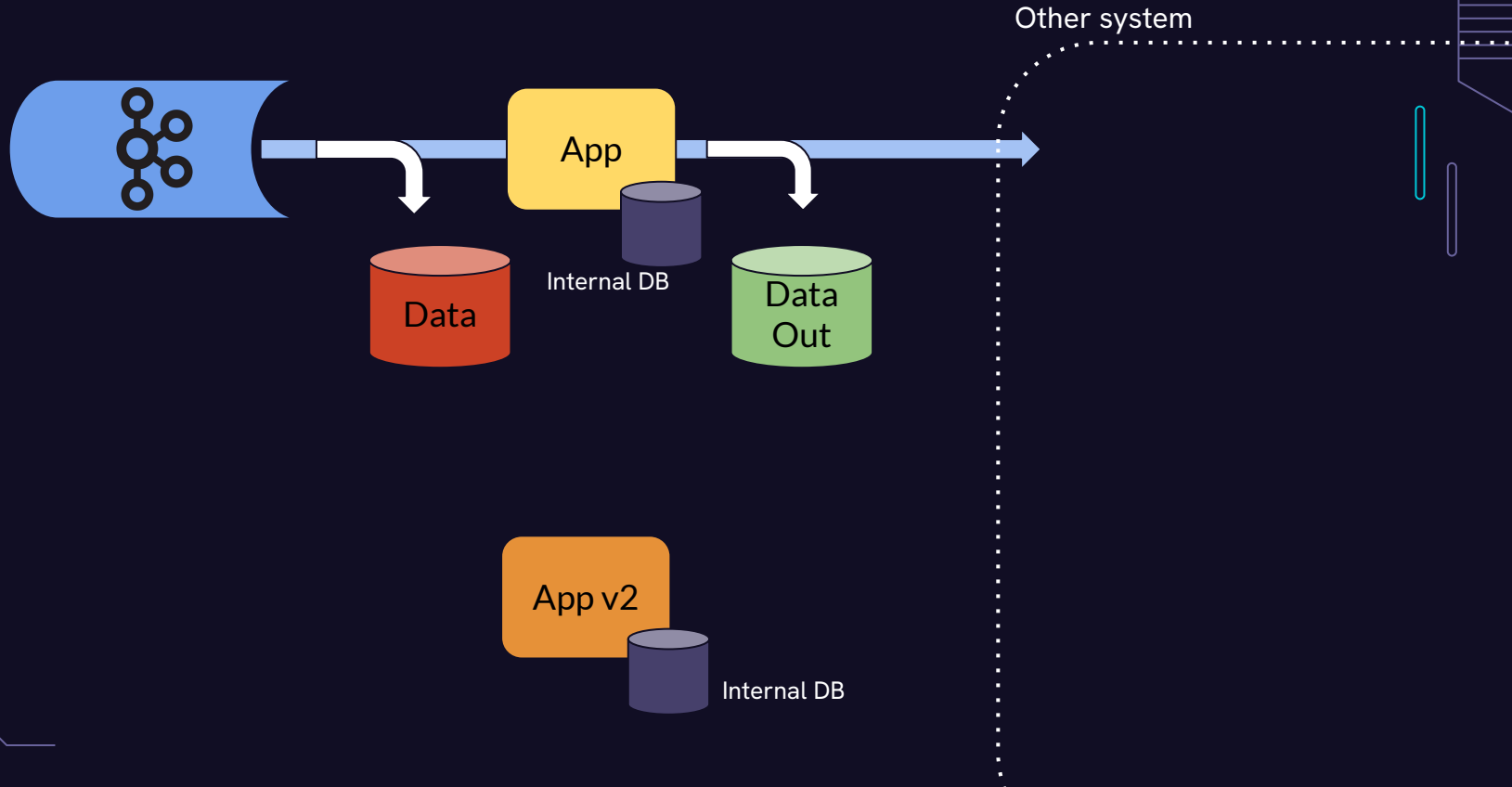
► What we want instead



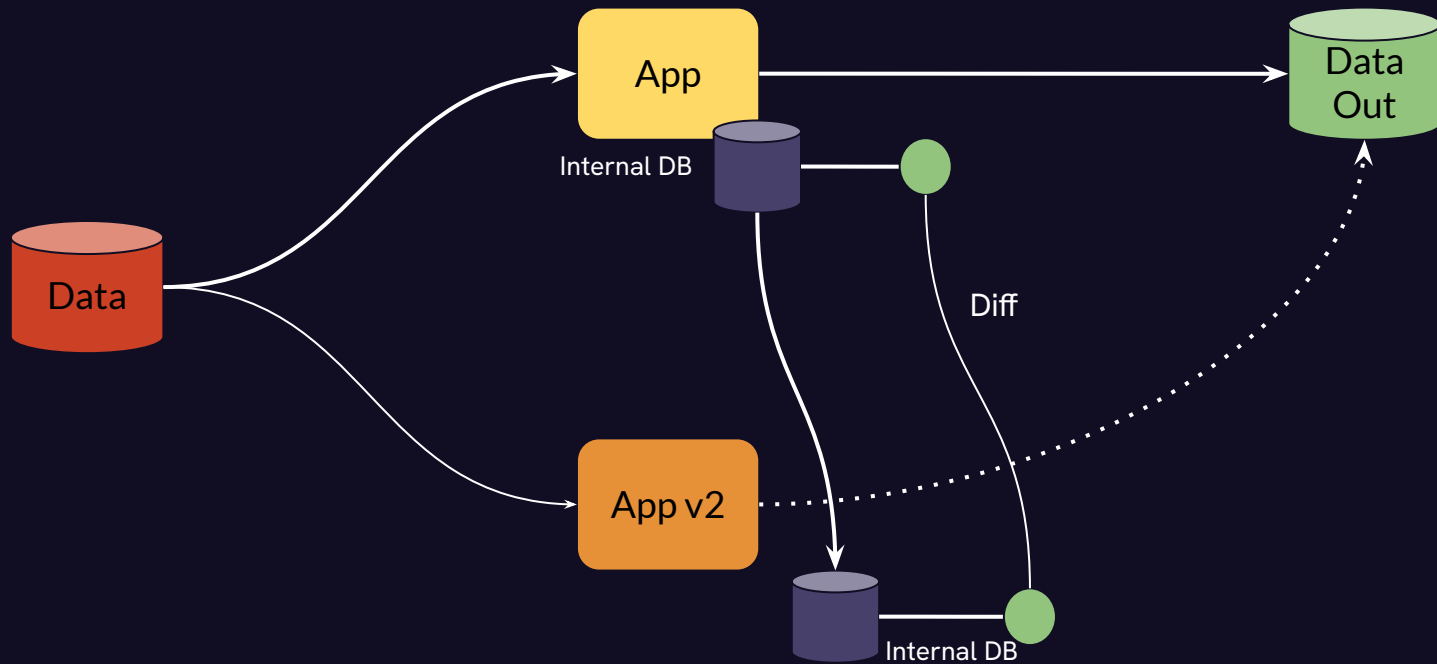
► A Different approach



► Typical Test scenario

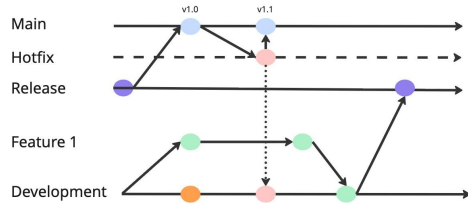


► What if ?

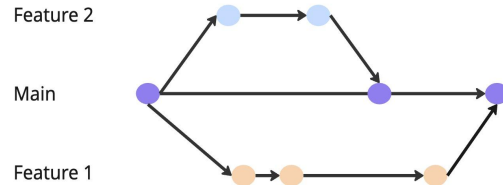


► Development methodologies

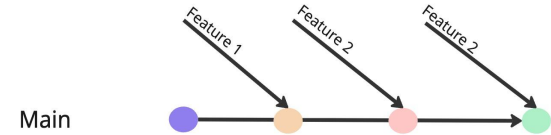
Git flow



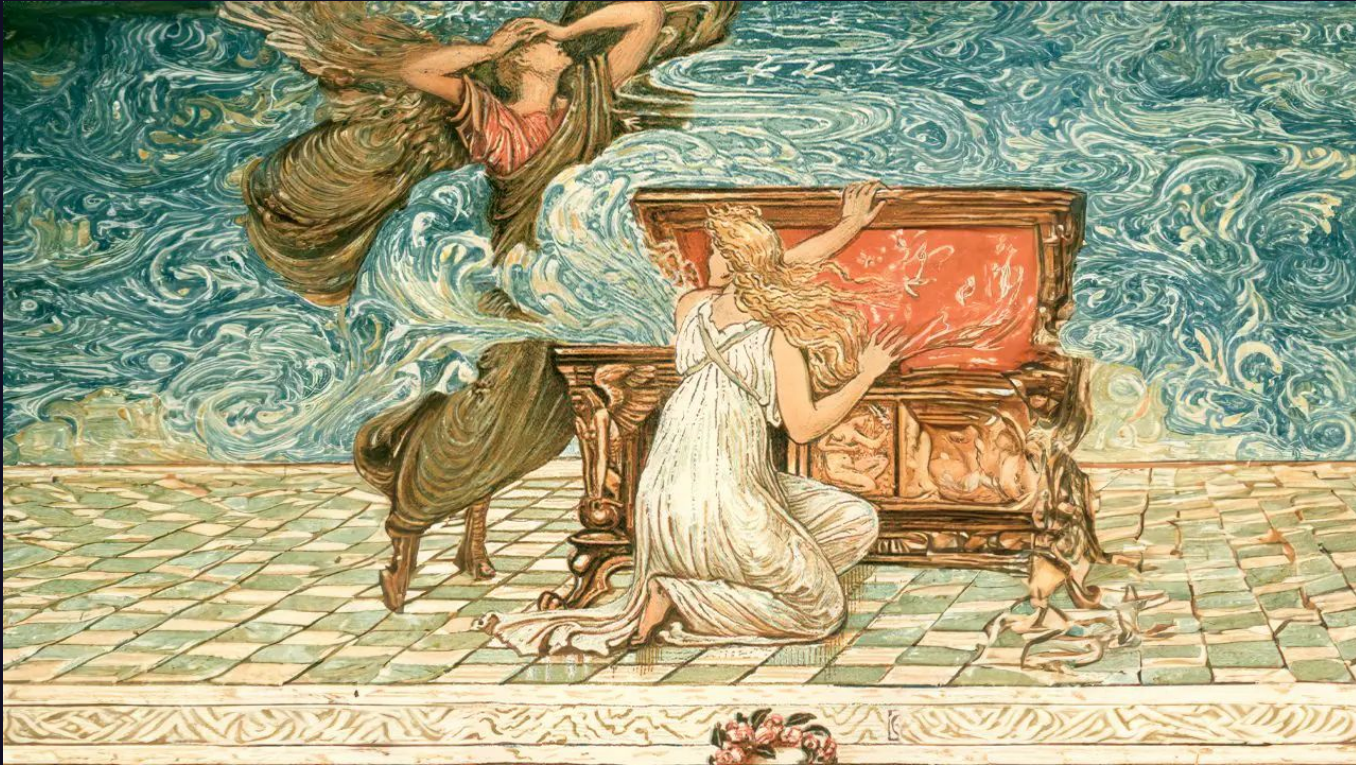
Github flow



Trunk based-development

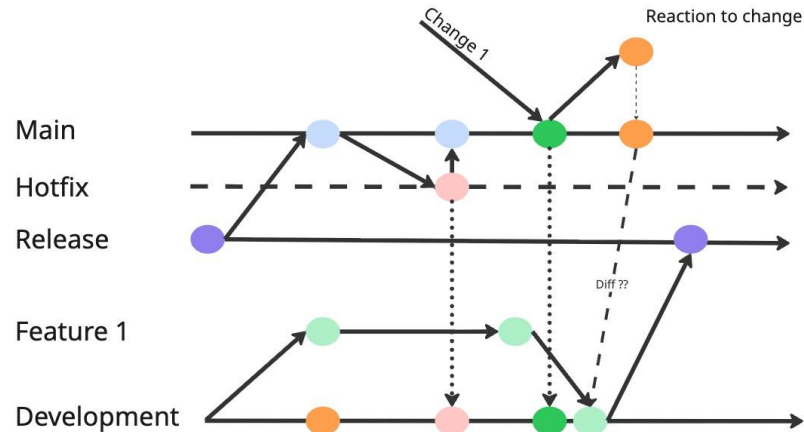


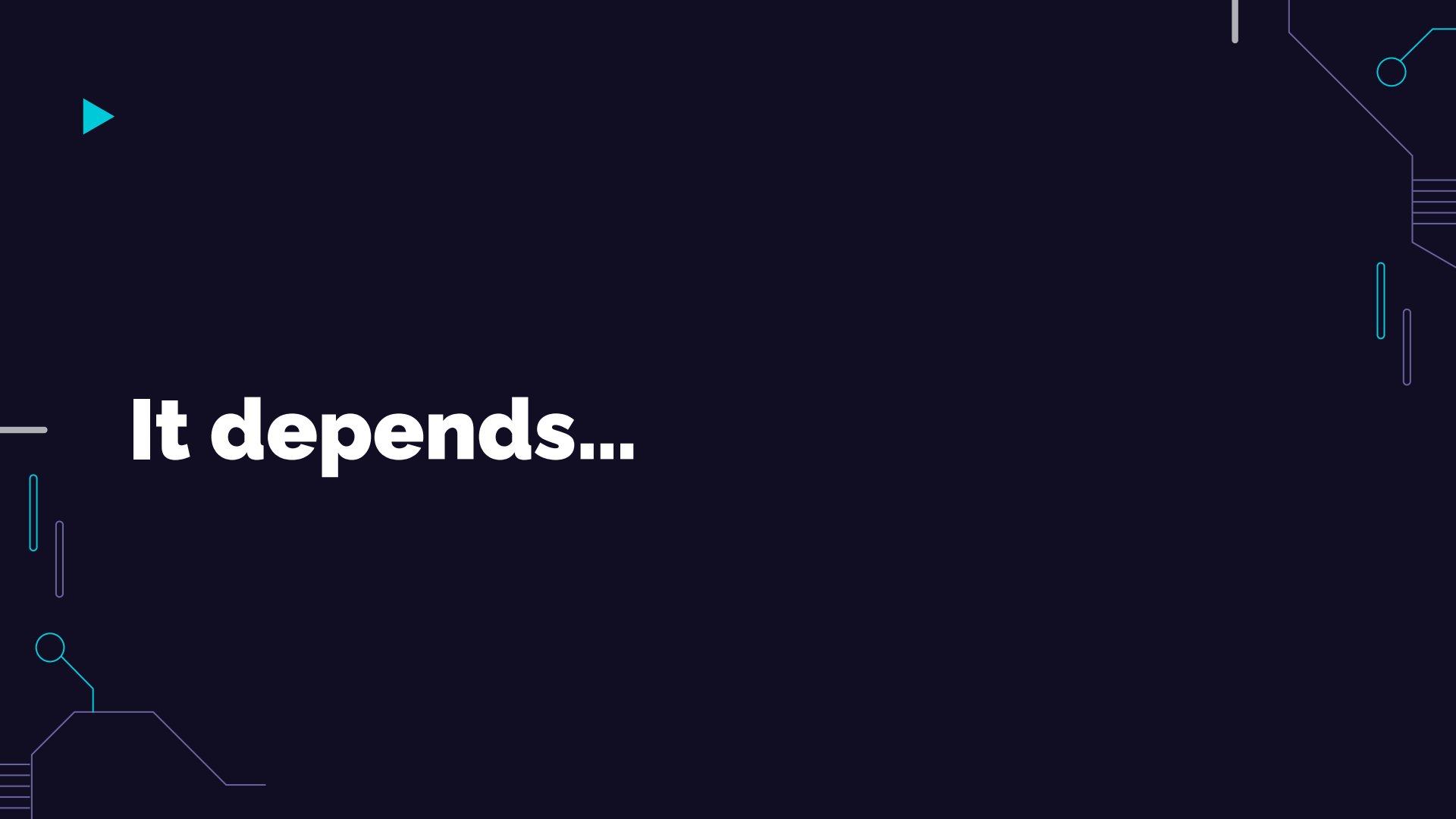
► Development versioning pattern for Data



► How we should adapt code methodologies to data?

- Data != Code: Data versioning listen to events and environment/status, code only to evolution of itself
- Data can change in production
- How we should compare different env?
- What semantic versioning should look like for data?



The background is a solid dark blue. In the top left, there is a small teal triangle pointing right. In the top right, there are several thin white lines forming a geometric pattern, including a circle and a series of horizontal lines. In the bottom left, there are two vertical teal rectangles and a white line forming a geometric shape. In the bottom right, there are two vertical teal rectangles and a white line forming a geometric shape.

It depends...

► **Thank you for your attention :)**